Claude Certified Architect Foundations

Score Report

CANDIDATE: Abhishek Yadav

CANDIDATE ID: ANTH455540

REGISTRATION NUMBER: 544205284

DATE: Sunday, August 16, 2026

SITE NUMBER: Proctored

EXAM CODE: CCAR-F

EXAM: Claude Certified Architect Foundations

This report shows the percentage of items in each section you answered correctly for the Claude Certified Architect Foundations

PASSING SCORE: 720 **YOUR SCORE: 958** **GRADE: Pass**

TEST OBJECTIVES	PERCENT CORRECT
Configure agentic loops to emit multiple tool calls within a single response turn, enabling parallel execution of independent subtasks and reducing total round-trip latency.	100%
Apply session resumption techniques—including targeted re-analysis of changed files and context injection—to restore agent state accurately without repeating prior work.	0%
Construct subagent prompts that include all findings, structured data, and source metadata required for task completion without returning to the coordinator for missing context.	100%
Evaluate subagent delegation strategies—goal-oriented versus procedural instructions—and select the approach that enables adaptive behavior while maintaining coordinator visibility and control.	100%
Configure subagent invocations with appropriate tool restrictions, context scoping, and system prompts that constrain each agent to its designated role.	100%
Diagnose misconfigured subagent spawning by identifying missing tool permissions, incorrect AgentDefinition parameters, or absent coordinator-to-subagent wiring.	100%
Decompose complex tasks into dynamically generated subtasks that adapt as new information is discovered, rather than executing a fixed sequence regardless of intermediate findings.	100%
Select the appropriate agentic review architecture—plan mode, direct execution, or multi-phase workflow—based on task scope, risk level, and human approval requirements.	100%
Design state persistence strategies for multi-agent pipelines that enable reliable resumption after interruption without repeating completed work or losing prior findings.	100%
Evaluate multi-agent orchestration patterns—coordinator-worker, parallel execution, and sequential pipelines—to select the structure best satisfying research coverage, latency, and reliability requirements.	100%

Configure Claude Code CLI invocations for automated CI/CD pipelines using non-interactive flags, permission modes, and cost and turn limits that prevent runaway executions.	100%
Structure iterative refinement workflows by providing concrete input-output examples, targeted feedback on specific failures, and batched issue descriptions for consolidated evaluation.	100%
Design Claude Code review configurations that load the correct project standards, restrict unnecessary tool access, and produce structured output suitable for automated downstream processing.	0%
Apply the context: fork frontmatter option to Skill and slash command configurations that should execute in an isolated subagent context, preventing cross-contamination of session state.	100%
Improve automated test generation quality by providing existing test files as context, defining fixture conventions, and specifying criteria that distinguish meaningful behavioral tests from trivial assertions.	100%
Select the correct Claude Code configuration mechanism—CLAUDE.md, .claude/rules/ with glob patterns, Skills, hooks, or settings permissions —based on guidance type and when it should apply.	100%
Apply systematic codebase exploration strategies using Grep, Glob, and Read tools that build incremental understanding while managing context window constraints.	100%
Design subagent output schemas that render structured data, prose summaries, and citation metadata in the format best suited to downstream synthesis and reporting.	100%
Design synthesis agent behavior that preserves source-level uncertainty, distinguishing well-established findings from contested claims rather than collapsing conflicting data into single confident statements.	100%
Apply context management strategies—subagent isolation, scratchpad files, and targeted file reading—to sustain coherent codebase exploration across sessions exceeding context limits.	100%
Apply context window optimization techniques—summarization, sliding windows, structured state objects, and selective retention—to maintain response quality as conversations exceed practical token limits.	100%
Design human review routing strategies that direct extractions to reviewers based on confidence scores, document characteristics, and field-level ambiguity rather than random sampling.	100%
Select the appropriate API processing mode—synchronous Messages API or asynchronous Message Batches API—based on latency requirements, workflow blocking behavior, and acceptable processing windows.	100%
Design specialized review passes that separate concerns—security, business logic, API design—into focused prompts with dedicated few-shot examples, preventing recall trade-offs from competing concerns in a single prompt.	100%
Select and implement the most reliable structured output method—tool use with JSON schema, prompt-based formatting, or prefilled responses —based on required schema compliance strictness.	0%
Design extraction schemas with optional fields, nullable values, and appropriate enum definitions that allow the model to accurately represent missing or ambiguous information without fabricating values.	100%
Implement tool use with defined JSON schemas to enforce structured output compliance, and configure tool_choice to guarantee tool invocation when conversational responses would cause downstream failures.	100%
Resolve structured output truncation failures by splitting large review tasks into smaller scoped API calls and merging resulting data structures, rather than increasing max_tokens beyond practical limits.	100%
Apply extraction accuracy patterns—structured schemas with optional fields, format normalization instructions, and few-shot examples—to reduce hallucination and improve consistency across varied document formats.	100%
Reduce automated review false positive rates by supplying project-specific conventions, accepted patterns, and exclusion criteria as persistent context applied on every review.	100%

Design prompt criteria that define explicit inclusion and exclusion boundaries, preventing the model from generating findings or extractions in categories where performance is unreliable.	100%
Select the appropriate Claude Code built-in tool—Grep, Glob, Read, or Bash—based on the nature of the search or file operation required for a given codebase task.	100%
Configure tool distribution in multi-agent systems by assigning each subagent only the tools required for its designated role, reducing decision complexity and preventing out-of-role tool invocations.	100%
Distinguish between MCP resources and tools, and expose server content as resources to reduce exploratory tool calls and improve agent efficiency in cross-system queries.	100%
Integrate MCP servers into Claude Code and agent applications by selecting the correct server scope, configuring authentication via environment variable expansion, and verifying tool discovery.	100%
Write MCP tool descriptions that clearly distinguish each tool's purpose, input formats, use-case boundaries, and relationships to semantically similar tools, reducing misrouting and incorrect tool selection.	100%
Configure the tool_choice parameter to guarantee tool invocation when structured output is required, and sequence multi-tool workflows so prerequisite data is obtained before dependent tools are called.	100%